Práctica 1 - Salida y explicación ejercicio a ejercicio

Este documento recoge la salida de la ejecución completa del proyecto (`python main.py`) y explica, ejercicio a ejercicio, qué hace cada paso y qué significan los resultados. Los datos llegan hasta el 9/11/2026, por lo que las cifras cambian si se vuelve a ejecutar otro día.

Resumen

Se ha construido un data lake por capas (bronze, silver y gold) con Delta Lake: precios de las 35 empresas del IBEX-35 (Yahoo Finance) cruzados con el sentimiento de un analista por sector, y exportados a Parquet para Power BI.

Dato	Valor
Empresas	35
Filas de precios (bronze y silver)	18.830
Periodo de precios	01/10/2024 a 09/11/2026
Filas del CSV de sentimiento	14.652
Sectores de sentimiento (tras limpiar)	12
Filas del hecho (gold)	18.830
Días distintos con cotización (dim_fecha)	538

Las capas son: **bronze** (datos tal cual llegan), **silver** (datos limpios y con tipos correctos) y **gold** (modelo en estrella listo para analizar).

Ej1: descarga de precios y Parquet

Ej1a. Se descarga el histórico de una sola acción (BBVA) y se define el esquema a mano. La salida es el esquema: `Date` (date), `Open` , `High` , `Low` , `Close` (double), `Volume` (long), `Ticker` (string), `Dividends` y `Stock_Splits` (double). Se usa `Stock_Splits` con guión bajo porque Parquet y Delta no admiten espacios en los nombres de columna. `Volume` es `long` para evitar desbordamientos con volúmenes grandes. En Ej1a aparecen `nullable = false` en `Date` , `Ticker` , `Dividends` y `Stock_Splits` : son columnas que siempre tienen valor.

Ej1b. Se descargan las 35 empresas y se guardan en `bronze/ibex` en formato Parquet. No imprime datos, solo escribe el fichero.

Ej1c. Se vuelve a leer el Parquet. Dos cosas a observar:

- Al leer, todas las columnas pasan a `nullable = true` . Es normal: Parquet no conserva la obligatoriedad al leerse.

- Se piden solo `Ticker`, `Date` y `Close` (**column pruning**). Parquet guarda cada columna en su propio bloque, así que Spark lee solo esos 3 y se salta los otros 6. Resultado: menos lectura de disco y menos memoria. Con un CSV habría que leer las filas completas.

Las 5 filas mostradas son de BBVA (27/07/2026 a 31/07/2026): se ven los precios con muchos decimales (por ejemplo 23.049999237060547) porque Yahoo los entrega en `float32`; esto se corrige en silver (Ej9).

Ej2: CSV de sentimiento y append en Parquet

Ej2a. Se lee el CSV del analista (separador `;`, sin `inferSchema`). Las tres columnas (`Fecha`, `Sector`, `Sentimiento`) llegan como `string`, porque un CSV es texto plano y no guarda tipos. Las 5 filas mostradas ya enseñan el problema de calidad: el sector viene escrito de formas distintas (`crypto`, `ENERGY`, `financial services` ...).

Ej2b. Se guarda en `bronze/sentimiento`. Se leen 14.652 filas y se escriben 14.652: no se pierde nada.

Ej2c. Se hace un `append` del mismo contenido: antes había 14.652 filas y después 29.304, el doble. Parquet es inmutable: `append` solo añade ficheros nuevos sin comprobar si esas filas ya existían, y no tiene claves ni transacciones. Por eso la carga no es idempotente (cargarla dos veces duplica los datos). Delta lo resuelve con `MERGE` (Ej4). Después se restaura la tabla con `overwrite`.

Ej3 y Ej4: Delta y MERGE

Ej3. Las dos tablas de bronze (`bronze/ibex` y `bronze/sentimiento`) se convierten a Delta sin reescribir los datos. El historial muestra una sola versión, la 0, con la operación `CONVERT` (420 ficheros Parquet convertidos en `bronze/ibex` y 1 en `bronze/sentimiento`). Ha aparecido la carpeta `_delta_log`: los ficheros Parquet no se han tocado, solo se han registrado en el log. A partir de ahora cada cambio añade un JSON nuevo, y recorrerlos permite reconstruir el estado de la tabla o volver a una versión anterior.

Ej4. Se descargan los datos nuevos hasta hoy y se insertan con `MERGE` usando la clave `(Ticker, Date)`:

	Filas antes	Filas después	Insertadas	Versión
MERGE 1	17.885	18.830	945	1
MERGE 2	18.830	18.830	0	1

En el primer MERGE entran los días nuevos. En el segundo no hay nada que insertar porque ya existen todos, así que **cargar dos veces no duplica datos** (con `append` sí, como se vio en el Ej2c). Por eso la versión no cambia en el segundo MERGE: al no modificarse nada no se crea versión nueva.

Ej5 y Ej6: schema enforcement, DELETE, historial y RESTORE

Ej5a (schema enforcement). Se intenta añadir un DataFrame con una columna que la tabla no tiene (`Close_mas5`). Delta lo rechaza con un error de *schema mismatch* y la tabla queda intacta: versión 1 y 18.830 filas antes y después.

Ej5b (mergeSchema). Con `mergeSchema=true` Delta acepta la escritura y añade la columna `Close_mas5` al esquema. La tabla pasa a la versión 2 y a 37.660 filas: se ha añadido la tabla entera otra vez, por eso las filas se duplican. Esto es esperable y se deshace en el Ej6.

Ej5c (DELETE). Se borran las filas de BBVA: tiene 1.076 filas antes y 0 después. DELETE crea una versión nueva en el log, pero los ficheros no se eliminan del disco hasta ejecutar un `VACUUM` , por eso el *time travel* todavía puede recuperar esas filas.

Ej5d (historial y time travel). El historial tiene 4 versiones:

Versión	Operación	Qué pasó
0	CONVERT	Conversión a Delta (Ej3)
1	MERGE	Datos nuevos (Ej4)
2	WRITE	Añadir `Close_mas5` (Ej5b)
3	DELETE	Borrar BBVA (Ej5c)

Leyendo con `versionAsOf 1` salen 18.830 filas y 9 columnas (el estado del Ej4). La tabla actual tiene 36.584 filas y 10 columnas: una columna más, filas duplicadas por el Ej5b y sin BBVA por el Ej5c. `versionAsOf` solo lee: la tabla actual no cambia.

Ej6 (RESTORE). Antes: versión 3, 36.584 filas, 10 columnas. Después de restaurar a la versión 1: versión 4, 18.830 filas, 9 columnas. RESTORE no borra el historial: añade la versión 4 (`RESTORE`) encima de las anteriores, así que se puede seguir viajando en el tiempo.

Ej7: perfil de los datos y API de Yahoo

Antes de limpiar hay que conocer los datos. El ejercicio saca un perfil de cada fuente.

Precios (`bronze/ibex`). La tabla está limpia: 18.830 filas, 35 empresas, de 01/10/2024 a 09/11/2026, sin duplicados por `(Ticker, Date)` , sin nulos, sin filas con `High < Low` y sin `Close <= 0` . El único problema son los decimales de `float32` que se vieron en el Ej1.

Sentimiento (`bronze/sentimiento`). Aquí está el problema real:

- Todo es texto (`Fecha` , `Sector` y `Sentimiento`).
- Hay 14.652 filas, de 2021-10-04 a 2026-09-30, sin nulos, sin coma decimal y sin duplicados.
- **El sector está escrito de muchas formas.** En el recuento por sector aparecen más de 40 variantes (la salida solo muestra las 40 primeras) para solo 12 sectores: mayúsculas (`BASIC MATERIALS`), minúsculas (`basic materials`), mayúscula inicial (`Basic Materials`) y con

espacios delante y detrás. Si no se normaliza, el mismo sector se contaría como varios y el cruce con los precios fallaría. Se arregla en el Ej8.

Datos que ofrece Yahoo (BBVA.MC). El objeto `info` tiene 164 campos; se muestran los más útiles: nombre, `sector`, `industry`, `country`, capitalización, `beta`, ratios, recomendación de analistas y precio objetivo. De aquí sale el sector de cada empresa, que no viene en el histórico de precios y es la clave para cruzar con el sentimiento.

También se consultan otras tablas de la API: `recommendations` (4 filas), `analyst_price_targets` (5 claves), `calendar` (8 claves) y el histórico del índice `^IBEX` (22 filas). `upgrades_downgrades` sale vacía (0 filas). La línea `HTTP Error 404` es de otra tabla que Yahoo no tiene para BBVA (datos fundamentales): el código lo controla y sigue, no es un fallo del programa.

La reflexión sobre cómo incorporar la información del experto con estos datos va en el PDF de la entrega.

Ej8 y Ej9: capa silver

Ej8 (silver del sentimiento). Se transforma `bronze/sentimiento` y se guarda en `silver/sentimiento`:

- `Fecha` pasa de texto a `date`.
- `Sentimiento` pasa de texto a `double` (cambiando la coma decimal por punto por si llegara así).
- `Sector` se normaliza (sin espacios y con mayúscula inicial en cada palabra), así coincide con el formato de Yahoo.
- Se quitan nulos y duplicados por `(Fecha, Sector)`.

Resultado: 14.652 filas en bronze y 14.652 en silver, porque el CSV no tenía nulos ni duplicados. El recuento por sector pasa de más de 40 variantes a 12 sectores: 1.276 días por sector, con dos excepciones. **Consumer Defensive** tiene solo 615 porque el experto empezó a puntuarlo más tarde (2024-05-07), y **Crypto** tiene 1.277.

Ej9 (silver de precios y empresas). Dos partes:

1. **Precios:** `bronze/ibex` pasa a `silver/ibex`. `Date` se renombra a `Fecha`, los precios y dividendos se redondean a 4 decimales (desaparece el ruido de `float32`: 23.049999237060547 pasa a 23.05) y se descartan sesiones incompletas o incoherentes y duplicados. Las filas no cambian: 18.830 en bronze y 18.830 en silver, porque los datos ya estaban limpios.
2. **Empresas:** el sector no está en los precios, así que se descarga del objeto `info` de Yahoo para cada empresa. Se guarda primero en crudo (`bronze/empresas`) y después limpio en `silver/empresas`, con columnas renombradas (`Nombre`, `Industria`, `Pais`, `Capitalizacion` ...) y una columna nueva `Sector_ES` con el sector en español (por ejemplo `Industrials → Industriales`).

Los dos esquemas mostrados confirman los tipos finales: fecha como `date`, precios como `double`, `Volume` como `long`.

Ej10 y Ej11: capa gold y anti-joins

Ej10 (modelo en estrella). Se construyen tres tablas en `gold`:

Tabla	Filas	Qué contiene
`hecho_cotizacion`	18.830	Una fila por empresa y día: precios, volumen, dividendos y sentimiento
`dim_empresa`	35	Una fila por empresa: nombre, sector, industria, país...
`dim_fecha`	538	Una fila por día con cotización: año, trimestre, mes, semana

El hecho tiene las mismas filas que `silver/ibex` (18.830 / 18.830): el cruce es un **left join**, así que no se pierde ninguna cotización. El sentimiento se une por `Fecha` y por el sector de la empresa; si ese día no hay sentimiento, queda nulo. Las dimensiones aportan los atributos para filtrar y agrupar en Power BI. En las 5 filas mostradas (ACS.MC, octubre de 2024) ya se ve el sentimiento unido a cada precio.

Ej11 (anti-joins). Un anti-join devuelve las filas de una tabla que **no** tienen pareja en la otra. Sirve para saber qué datos no cruzan.

(a) Cotizaciones sin sentimiento: 980 filas. Son todas posteriores al 30/09/2026, último día del CSV del experto (el resumen muestra desde 2026-10-01 hasta 2026-11-09 en todos los sectores). Los precios llegan hasta hoy, pero el experto dejó de puntuar el 30/09.

(b) Sentimiento sin cotizaciones: 9.042 filas. Tiene dos causas:

- **Fechas anteriores a los precios.** El CSV empieza en 2021 y los precios en octubre de 2024, así que casi todos los sectores tienen 766 filas hasta 2024-09-30 (Consumer Defensive, 105 desde 2024-05-07).
- **El sector Crypto.** Tiene 1.277 filas, de 2021 a 2026, porque no es un sector de Yahoo y ninguna empresa del IBEX pertenece a él.

Solución: no se pierde nada. El left join del Ej10 conserva las cotizaciones sin sentimiento (nulo, filtrable en el dashboard) y el sentimiento sin precios sigue en `silver/sentimiento`. No se rellenan los huecos arrastrando el último valor porque sería inventar la opinión del experto.

Ej12 y Ej13: indicadores y exportación

Ej12 (columnas calculadas). Se añaden al hecho cuatro columnas, calculadas por empresa y en orden de fecha (con funciones de ventana):

Columna	Qué mide
`Variacion_Diaria_Pct`	Cambio porcentual del cierre respecto al día anterior
`Rango_Intradia_Pct`	Diferencia entre máximo y mínimo del día, en % del cierre
`Variacion_Acumulada_Pct`	Cambio porcentual del cierre desde la primera sesión de esa empresa

Columna	Qué mide
`Media_Movil_20`	Media del cierre de las últimas 20 sesiones

Los nulos por columna son una buena comprobación y todos tienen explicación:

- `Sentimiento` : 980, justo las cotizaciones sin sentimiento del Ej11(a).
- `Variacion_Diaria_Pct` : 35, la primera sesión de cada empresa, que no tiene día anterior con el que comparar.
- `Media_Movil_20` : 665, que son 35 × 19: las 19 primeras sesiones de cada empresa no tienen 20 datos todavía, y no se calcula una media con menos.
- El resto de columnas: 0 nulos.

La muestra es BBVA en noviembre de 2026. Por ejemplo, el 2026-11-04 el cierre es 22,44 y sube un 4,32 % respecto al día anterior (21,51). La variación acumulada de 141,29 % significa que el precio es 2,4 veces el de la primera sesión (octubre de 2024). Se guarda como una versión nueva de Delta, así que el historial se conserva.

Ej13 (exportación). Antes de exportar se comprueba la integridad de la estrella con cinco controles, y todos dan 0:

- Tickers del hecho sin fila en `dim_empresa` .
- Fechas del hecho sin fila en `dim_fecha` .
- Tickers repetidos en `dim_empresa` .
- Fechas repetidas en `dim_fecha` .
- Pares `(Ticker, Fecha)` repetidos en el hecho.

Con eso se garantiza que no hay claves huérfanas ni repetidas. Después se exportan las tres tablas a `data/export/gold` , cada una en su carpeta con un único fichero Parquet, ordenado por su clave y comprimido con Snappy. Se relee lo exportado y las filas coinciden: 18.830 / 18.830 en el hecho, 35 / 35 en `dim_empresa` y 538 / 538 en `dim_fecha` . El esquema del hecho exportado tiene 14 columnas con tipos simples (`date` , `double` , `long` y `string`) que Power BI lee sin conversiones; `Fecha` es de tipo `date` , necesario para la tabla de fechas de Power BI.

Declaración de uso de IA

He utilizado Claude (Anthropic) como asistente durante la práctica: para revisar y simplificar el código, y para redactar y organizar este documento de explicación a partir de la salida del programa. Las funciones empleadas son las vistas en clase (Práctica 0 y Práctica 1) y el código se ha ejecutado y comprobado en mi equipo.